

```

    If condition satisfies → return 1;

    Else → return 0;

recCall1= f(parameters);
recCall2 = f(parameters);
.....
nRecCall= f(parameters);

return recCall1+ recCall1 +..... + nRecCall;
}
```

Prefix Sum Problems

Prefix sum problems calculate ranges and find the **maximum occuring instance** among these. Some problems having these -

- Maximum platforms required when arrival and departure time of trains are given.
- Maximum population - [LC - 1845 Maximum Population Year](#)

```
//logs = [[1950,1961],[1960,1971],[1970,1981]]

int[] year = new int[101];
    // O(n) -> n is log.length

for(int[] log : logs){
    year[log[0] - 1950]++;
    year[log[1] - 1950]--;
}

int maxNum = year[0], maxYear = 1950;

for(int i = 1; i < year.length; i++){
    year[i] += year[i - 1]; // Generating Prefix Sum
    if(year[i] > maxNum){
        maxNum = year[i];
        maxYear = i + 1950;
    }
}
return maxYear;
```

Binary Search

Search on monotonic functions.

1. First and last occurrences for a given ‘target’ value.

```
class Solution {
    public int[] searchRange(int[] nums, int target) {
        int low = 0;
        int high = nums.length - 1;

        //first index
```

```

int firstIdx = -1;
while(high >= low){
    int mid = low + (high - low) /2;
    if(nums[mid] == target){
        firstIdx = mid; //save probable answer.
        high = mid - 1; //search in left-space for more occurrence.
    } else if(nums[mid] < target){
        low = mid +1;
    } else {
        high = mid - 1;
    }
}

// Last index;
low = 0 ; high = nums.length - 1;
int lastIdx = -1;
while(high >= low){
    int mid = low + (high - low) /2;
    if(nums[mid] == target){
        lastIdx = mid; //save probable answer.
        low = mid + 1; //search in right-space for more occurrence.
    } else if(nums[mid] < target){
        low = mid +1;
    } else {
        high = mid - 1;
    }
}
int[] ans = {firstIdx, lastIdx};
return(ans);
}
}

```

3. Search in a Rotated Sorted Array the given target value.

Intuition: one half of the array is always sorted,

Elimination of search space should be done **after identifying the search space** .

```

class Solution {
    public int search(int[] nums, int target) {
        // [4,5,6,7,0,1,2,3]

        int low = 0;
        int high = nums.length-1;
        int ans;
        while(high >= low){
            int mid = low + (high - low) / 2;
            if(nums[mid] == target)
                return mid;

            // identification of search space. // Check occurrence in SORTED SPACE only.
            // algo will take care in non-sorted space.
            //if left side is sorted
            if(nums[low] <= nums[mid]){
                //check if target lies in this search space

                if(nums[low] <= target && nums[mid] >= target)
                    high = mid - 1;
                else
                    low = mid + 1;
            }
        }
    }
}

```

```

        } else {
            //right side is sorted, since one of the sides is always sorted : observation
            //check if target lies in this search space
            if(nums[mid] <= target && nums[high] >= target)
                low = mid + 1;
            else
                high = mid - 1;
        }

    }

    return -1;

}
}

```

4. Variant : search element in an array if this array contains duplicates also eg .
[3,3,3,1,3,3,3,3,3,3,3,3]

```

class Solution {
    public boolean search(int[] nums, int target) {
        // [4,5,6,6,7,0,1,2,4,4]
        int low = 0;
        int high = nums.length - 1;

        while(high >= low){
            int mid = low + (high - low) / 2;
            if(nums[mid] == target)
                return true;

            // shrink search space if nums[low] == nums[mid] == nums[high]
            //eg. case - [3,3,3,1,3,3,3,3,3,3,3,3]

            //everything -else remains the same.
            if(nums[low] == nums[mid] && nums[mid] == nums[high]){
                low++;
                high--;
                continue;
            }

            // if left search space sorted
            if(nums[low] <= nums[mid]){
                if(nums[low] <= target && target < nums[mid])
                    high = mid - 1;
                else
                    low = mid + 1;
            } else {
                if(nums[mid] < target && target <= nums[high])
                    low = mid + 1;
                else
                    high = mid - 1;
            }
        }

        return false;
    }
}

```

5. Find Minimum in Rotated Sorted Array **IMP**

Intuition :

1. keep the minimum of sorted search space as a probable answer.
2. Check in the remaining unsorted array for further minimum value.

```
class Solution {
    public int findMin(int[] nums) {
        int low = 0;
        int high = nums.length - 1;

        int ans = nums[0];
        while(high >= low){
            int mid = low + (high - low) / 2;
            if(nums[low] <= nums[mid]){
                //left side is sorted and the starting value of it can be the answer.
                ans = Math.min(ans,nums[low]);

                //check in remaining unsorted array i.e right side
                low = mid +1;
            } else{
                //right side is sorted
                ans = Math.min(ans, nums[mid]);

                // check the unsorted left side.
                high = mid -1;
            }
        }

        return ans;
    }
}
```

6. Find any Local Maxima. Considering $nums[-1] = -\text{infinity}$ and $nums[n+1] = -\text{infinity}$.

Intuition :

The following picture explains how we select a search space that **guarantees to have a local maxima**.

case - 1:

○ → answer

○
mid

if arr[mid] < arr[mid + 1]

this guarantees that right side is either monotonically increasing or it contains a local maxima.

case - 2:

○ → local maxima : ans

○
mid

if arr[mid] < arr[mid - 1]

this guarantees that left side is either monotonically decreasing or contains a local maxima.

```
class Solution {
    public int findPeakElement(int[] nums) {
        // RETURN index of peak.
        int n = nums.length;
        if(n == 1)
            return 0;

        //base case : either start or last element.
        if(nums[0] > nums[1])
            return 0;
        if(nums[n-1] > nums[n-2])
            return n-1;

        int low = 1;
        int high = n - 2;
        while( high >= low){
            int mid = low + (high - low) / 2;

            // if mid greater than left and right neighbor then that's the answer.
            if(nums[mid] > nums[mid - 1] && nums[mid] > nums[mid + 1])
                return mid;

            //this guarantees that either right side is monotonically increasing -
            // in that case last element of search space is the answer
            // or will contain a local maxima. (inflection point is the answer)
            if(nums[mid] < nums[mid + 1]){
                low = mid + 1;
            } else{
                high = mid-1;
            }
        }

        return -1;
    }
}
```

7. BS to find Minimum of Maximum or Maximum of Minimum -

- Aggressive Cows, Magnetic Force b/w balls - Min-max prob,

To solve Min-Max problem → 2 functions are required

- Using BS -> we get the min dis b/w objects
- Check if the min-dis **places required number** of objects with min-distance.

```
class Solution {
    private boolean canPlaceBalls(int minDis,int[] position,int requiredBalls){
        int placedBalls = 1;
        int lastUsedIdx = position[0];
        for(int i=1; i<position.length; i++){
            if(position[i]-lastUsedIdx >= minDis){
                placedBalls++;
                lastUsedIdx = position[i];
            }
        }
        System.out.println(minDis + " : " + placedBalls);
        if(placedBalls < requiredBalls)
            return false;

        return true;
    }
    public int maxDistance(int[] position, int m) {
        Arrays.sort(position);
        int low = 1;    //min dis b/w balls;
        int high=position[position.length - 1] - position[0];    //max dis b/w balls.
        int minDis = -1; //we need to return the max value for it.
        while(high >= low){
            int dis = low + (high - low)/2;
            System.out.println(low + " : " + dis + " : " + high);
            // at a given distance how many balls can we put.
            //if greater balls are required, apply more space else reduce space.
            if(canPlaceBalls(dis,position,m) == true){
                minDis = Math.max(minDis,dis);
                low = dis + 1; //check with higher distance if possible
            }else{
                high = dis - 1;
            }
        }

        return minDis;
    }
}
```

- Maximum Pages allocated to students - Max-min prob

Goal - Maximum of subarray sum is minimum.


```
import java.util.ArrayList;

public class Solution {
    public static int findPages(ArrayList<Integer> arr, int n, int m) {
        // Write your code here.

        if(m > n)
            return -1;

        int low = arr.get(0), high=0;
        for(int x : arr){
            low = Math.max(low, x);
            high += x;
        }

        int ans = high;
        while(high >= low){

            int mid = low + (high - low)/2;
            int students = checkStudents(mid, arr);
            // System.out.println(low + " : " + mid + " : " + high);
            // System.out.println("students :" + students);
            if(students <= m){
                ans = Math.min(mid, ans);
                high = mid - 1;
            } else
                low = mid + 1;

        }

        return ans;
    }

    private static int checkStudents(int maxPages, ArrayList<Integer> arr){
        int count = 1;
        int pageSum = arr.get(0);
        for(int i = 1; i < arr.size(); i++){
            if(pageSum + arr.get(i) > maxPages){
                count++;
                pageSum = arr.get(i);
            } else{
                pageSum += arr.get(i);
            }
        }

        return count;
    }
}
```

8. Median of 2 sorted Arrays

1. Equal length arrays:

```
public double findMedianSortedArrays(int[] nums1, int[] nums2) {

    int N = nums1.length;
```

```

int low = 0;
int high = N;

while(low <= high){
    int mid1 = (low + high)/2;
    int mid2 = (N-1) - mid1;

    int l1 = mid1==0 ? Integer.MIN_VALUE : nums1[mid1-1];
    int r1 = mid1==N ? Integer.MAX_VALUE : nums1[mid1];

    int l2 = mid2==0 ? Integer.MIN_VALUE : nums2[mid2-1];
    int r2 = mid2==N? Integer.MAX_VALUE : nums2[mid2];

    //check valid partition
    if(l1 <= r2 && l2 <= r1 )
        return ((Math.max(l1,l2) + Math.min(r1,r2))/2.0);

    //trim from left array
    if(l1 > l2)
        high = mid1 - 1;
    else
        low = mid1 + 1;
}
return 1.0;
}

```

2. If len1 != len2

Make operation on smaller array only,

```

public static double median(int[] a, int[] b) {
    // Write your code here.

    int N = a.length;
    int M = b.length;

    if(N > M) //BS on shorter array
        return median(b, a);

    //considering N <= M
    int low = 0;
    int high = N;
    while( low <= high){
        int mid1 = (low + high)/2;
        int mid2 = (N + M + 1)/2 - mid1;

        int l1 = (mid1 == 0) ? Integer.MIN_VALUE : a[mid1 - 1];
        int r1 = (mid1 == N) ? Integer.MAX_VALUE : a[mid1];

        int l2 = (mid2 == 0) ? Integer.MIN_VALUE : b[mid2 - 1];
        int r2 = (mid2 == M) ? Integer.MAX_VALUE : b[mid2];

        if(l1 <= r2 && l2 <= r1){
            if((N + M )%2 == 0)
                return (Math.max(l1,l2) + Math.min(r1,r2))/2.0;

            return Math.max(l1,l2);
        }

        if(l1 > r2)
            high = mid1 - 1;
        else if (l2 > r1)
            low = mid1 + 1;
    }
}

```



```
    }

    return 0;
}
```

9. Median of Sorted Matrix.

8	25	30	35	47
6	19	46	54	89
23	45	56	76	90
12	14	34	55	67
7	23	56	67	99

Given a row wise sorted matrix. **Where n and m are odd.** Thus median is always (n*m / 2)

- Intuition :
- Ans range (min(arr), max(arr))
 - If **ans is greater than (n*m/2) then that's the required answer** (if present in matrix else next greater than that present in matrix) -
 - To get the number of elements in each row, we can apply binary search.

10. Kth Smallest element in matrix

- K-th smallest element ->2 approach
1. Create maxheap of max k size
 - easy but O(n*n * log N) : iterate over all elements to add in PQ
 2. Apply BS
 - O(N* logN)

Min ans -> ar[0][0]
Max ans -> ar[n-1][m-1]

Our answer lies b/w these two only.

```
class Solution {
    public int kthSmallest(int[][] matrix, int k) {
        int n = matrix.length;
        int m = matrix[0].length;

        int low = matrix[0][0];
        int high = matrix[n-1][m-1];

        while(low < high){
            int mid = low + (high - low)/2;
            int count = countLessEqual(matrix, mid);

            if(count < k)
                low = mid + 1;
            else
                high = mid;
        }
        return low;
    }
}
```

```
int countLessEqual(int[][] matrix, int target){
    int n = matrix.length;
    int count = 0;
    int row = n-1, col = 0;

    while(row >= 0 && col < n){
        if(matrix[row][col] <= target){
            count += (row + 1);
            col++;
        } else{
            row--;
        }
    }

    return count;
}
```

Sliding Window & Two pointers

O(n^2) to generate all possible substring:

```
for (int i = 0; i < n; i++)
{
    for (int j = i; j < n; j++)
    {
        substrings.add(str.substring(i, j + 1));
    }
}
```

Note : problems involving Maximum substring stuff -> gives **hint** of Two pointer and Sliding window.*

1. Longest substring with no repeating. [LC -3](#)

class Solution {

```
public int lengthOfLongestSubstring(String s) {
    int[] indexInfo = new int[1000];
    Arrays.fill(indexInfo, -1);
    int end = 0, start = 0;

    int ans = 0;
    while(start < s.length()){
        //seeing this char first time
        if(indexInfo[(int)s.charAt(start)] == -1){
            ans = Math.max(ans,(start - end + 1)); //number of characters in that window
            indexInfo[(int)s.charAt(start)] = start;    //mark index where it was present.
            start++;
        } else {
            // already seen this character.
            int lastSeenPosition = indexInfo[(int)s.charAt(start)];
            while(end <= lastSeenPosition){
                //remove elements from window
                indexInfo[(int)s.charAt(end)] = -1;
            }
        }
    }
}
```